Stochastic Traffic Signal Timing Optimization

PBCBT504: Operations Research Project Report

Submitted by:

Athuljith S	**MDL24CSBS021**
Dijith Dinesh	**MDL24CSBS023**
Peter Joe Menachery	**MDL24CSBS048**
Pranav Manoj	**MDL24CSBS049**



Department of Computer Engineering

Govt. Model Engineering College, Thrikkakara

Ernakulam, Kochi 682021

https://www.mec.ac.in

October 2026

Model Engineering College, Thrikkakara

Department of Computer Engineering



CERTIFICATE

This is to certify that the project report entitled **"Stochastic Traffic Signal Timing Optimization"** is a record of the work carried out by

Athuljith S	MDL24CSBS021
Dijith Dinesh	MDL24CSBS023
Peter Joe Menachery	MDL24CSBS048
Pranav Manoj	MDL24CSBS049

Fifth Semester B.Tech Computer Science & Business Systems

students, in partial fulfillment of the requirements for the course **PBCBT504: Operations Research**.

The course forms part of the Bachelor of Technology programme of **APJ Abdul Kalam Technological University**.

Project Guide	**Head of the Department**
Signature and name	Signature and name

Project Coordinator

Signature and name

Date: _____

Acknowledgements

We thank the faculty of the Department of Computer Engineering, Government Model Engineering College, for their guidance in framing this as an Operations Research study. The course material on queues, stochastic arrivals, and simulation provided the academic basis for the work. We also thank those who reviewed the problem formulation and helped us distinguish a reproducible simulation result from a field-validated traffic recommendation.

Athuljith S	MDL24CSBS021
Dijith Dinesh	MDL24CSBS023
Peter Joe Menachery	MDL24CSBS048
Pranav Manoj	MDL24CSBS049

Model Engineering College

Abstract

This report studies a finite-horizon traffic signal timing problem using stochastic queuing simulation and constrained enumeration. Four incoming approaches each have dedicated left-turn, through, and right-turn queues. Vehicle arrivals follow independently configured homogeneous Poisson processes; turning movements are sampled at fixed probabilities. A seeded linear congruential generator supplies uniform pseudorandom values, and exponential inverse-transform sampling creates arrival timestamps. A discrete-event engine applies first-come, first-served service with two-second headways during four protected phases and no service during clearance.

The decision is the integer-second green allocation within a fixed 96-second cycle. The objective is expected cumulative waiting, measured as the integrated waiting-queue population across all twelve lanes in a specified operating period. Candidate allocations are evaluated on shared training traffic; the selected allocation and simple baselines are evaluated on separate, matched fresh scenarios. For hypothetical demand of 1,800 vehicles/hour and a ten-minute, empty-start horizon, a four-second-grid search selected 28/16/24/12 seconds for North/East/South/West. Its fresh mean waiting was 10,507.8 vehicle-seconds per run (pointwise 95% interval 10,430.7–10,584.9). The paired saving against the entered timing was 104.2 vehicle-seconds per run (95% interval 69.6–138.8). The selected plan equals a demand-proportional baseline on this grid.

A one-second-grid search gives a further but small 0.42% matched improvement. Heavy demand exceeds the modeled straight-lane capacity under every feasible reallocation, while a 32-minute comparison does not resolve the ten-minute timing advantage. These results are conditional on synthetic rates and simplified geometry; measured traffic data would be needed before a real-world recommendation.

Contents

Contents i

List of Figures iii

List of Tables iv

1 Introduction 1
- 1.1 Problem Statement 1
- 1.2 Context, Significance, and Objectives 1

2 System Study 2
- 2.1 Physical Scenario and Queue Structure 2
- 2.2 Queueing Assumptions and Data Source 2

3 Software Requirements Specification (SRS) 4
- 3.1 System and Software Requirements 4
- 3.2 Functional Requirements 4

4 System Design and Architecture 5
- 4.1 Mathematical Model of Arrivals 5
- 4.2 Discrete-Event State Machine 5
- 4.3 Objective and Complete Formulation 6
- 4.4 Performance Measures 6

5 Implementation 7
- 5.1 Rust/WASM Engine and Experiment Pipeline 7
- 5.2 Optimization and Application 8

6 Testing 9
- 6.1 Verification and Monte Carlo Convergence 9

7 Results and Discussion **10**
7.1 Reference Ten-Minute Result 10
7.2 Mean Queue Behavior 11
7.3 Fine-Grid Search and Runtime 11
7.4 Demand, Capacity, and Horizon Sensitivity 12

8 Conclusion and Future Scope **14**
8.1 Conclusion 14
8.2 Limitations 14
8.3 Future Scope 14

References **15**

List of Figures

5.1 Browser application: scenario controls (left) and signal/vehicle replay (right) 8

6.1 Running mean cumulative waiting over 1,000 fresh reference runs 9

7.1 Fresh mean waiting for selected and reference timings; lower is better 11

7.2 Average waiting queues by approach over the ten-minute period 11

List of Tables

2.1 Reference experiment inputs 2

7.1 Principal fresh-evaluation result, ten-minute reference case . . 10
7.2 Original-study native runtime for each principal study request 12
7.3 Re-optimized four-second-grid demand scenarios 13
7.4 Paired saving of 28/16/24/12 s against 24/16/24/16 s 13

Chapter 1

Introduction

1.1 Problem Statement

A fixed-cycle signal shares limited green time among competing approaches, while random arrivals make queues and waiting vary between runs. This study asks which feasible green allocation minimizes expected cumulative waiting over a finite operating period. It models a four-approach junction with dedicated turn lanes, empty initial queues, and a ten-minute primary horizon; it uses synthetic demand and excludes field calibration, pedestrians, and downstream spillback.

1.2 Context, Significance, and Objectives

Signal timing affects waiting, throughput, and queue persistence. Equal green can misallocate capacity when demand differs, while optimization may offer only a small gain over a demand-based rule. Operations Research supplies a disciplined way to compare constrained decisions using queueing and simulation [1, 2]. The project formulates this finite-horizon objective, implements a reproducible Rust discrete-event model, exhaustively compares feasible timings on selected grids, evaluates demand and horizon sensitivity, and presents the evidence and limitations in a browser application.

Chapter 2

System Study

2.1 Physical Scenario and Queue Structure

Four approaches (N/E/S/W) each have dedicated left, straight, and right FCFS queues. Turns exit onto the next road (from North: left/east, straight/south, right/west; same rotation elsewhere). Three lanes serve during their protected phase; clearance stops service. Capacity follows FHWA's green-share relation [4].

2.2 Queueing Assumptions and Data Source

M/M/1 assumes exponential service at a continuously available server; our deterministic signal-limited service differs, so its waiting formula does not apply directly [2, 1]. Little's formula needs compatible long-run measures [3]; we verify finite-run waiting by queue-area integration. The reference experiments use synthetic inputs and describe no named junction.

Table 2.1: Reference experiment inputs

Parameter	Value	Interpretation
Approach rates N/E/S/W	600/350/550/300 veh/h	Total 1,800 veh/h
Turn probabilities L/S/R	20/60/20%	Independently sampled
Cycle, clearance	96 s, 16 s	80 s green per cycle
Green bounds and grid	8–44 s, 4 s	415 feasible plans
Service, horizon	2 s; 600 s	Empty initial queues
Entered plan; seed	24/16/24/16 s; 2026	Reproducible reference

The application also includes a separate data-based demonstration preset from the City of Toronto's Multimodal Intersection Turning Movement Counts. The

City-sourced intersection count table lists 3,139 vehicles in the September 16, 2026 AM peak hour at Eglinton Avenue East / Victoria Park Avenue. The summary value is an intersection total, so the preset divides it equally among four approaches (784.75 vehicles/hour each) and retains the application's 20/60/20 turn probabilities. Those disaggregations are assumptions, not measured approach or movement counts. This preset is separate from, and was not used in, the reference results in this report. The source is licensed under the Open Government Licence – Toronto.

Chapter 3

Software Requirements Specification (SRS)

3.1 System and Software Requirements

The system accepts a junction scenario and timing-search request, simulates queues reproducibly, reports uncertainty, and displays interpretable OR results. The Rust core runs natively and as WebAssembly; SolidJS, SVG, and Vite Plus provide the browser interface. Batched runs expose progress, and multiple Web Workers can divide large candidate searches. The interface displays engine outputs without changing the mathematical result.

3.2 Functional Requirements

- Validate demand, turns, signal bounds, and feasible grids.
- Generate seeded arrivals and turns; simulate events with conservation and queue-area accounting.
- Enumerate feasible timings on common training scenarios and choose the lowest estimated waiting.
- Compare the selected plan with references on matched fresh scenarios and report uncertainty.
- Display the recommendation, units, queue plots, outcome spread, progress, and replay.

Identical requests and seeds reproduce results. Summary memory remains bounded as replications grow; the core is independent of rendering, and saved results retain requests and outputs for audit.

Chapter 4

System Design and Architecture

4.1 Mathematical Model of Arrivals

For approach a, let λ_a be vehicles per second. A homogeneous Poisson process has independent exponential inter-arrival times [1] and satisfies

$$N_a(t) \sim \text{Poisson}(\lambda_a t), \qquad ¶[N_a(t) = k] = \frac{(\lambda_a t)^k e^{-\lambda_a t}}{k!}. \tag{4.1}$$

Independent exponential gaps construct arrival times event by event. For $U \in (0, 1)$,

$$\Delta t = -\frac{\ln(1 - U)}{\lambda_a}. \tag{4.2}$$

At 600 vehicles/hour, $\lambda = 1/6$ vehicles/second and mean gap is six seconds. For $U = 0.20$, Equation 4.2 gives $-6 \ln(0.8) = 1.34$ seconds. A second uniform draw assigns a turn using the configured cumulative probabilities. Thus a 20/60/20 split is random, not a fixed count of vehicles.

The linear congruential generator uses

$$x_{n+1} = (6364136223846793005 x_n + 1442695040888963407) \bmod 2^{64}. \tag{4.3}$$

The upper 52 state bits produce a uniform value strictly inside $(0, 1)$. A full-period state recurrence does not prove independent random outputs; this is a reproducible simulation source with known limitations.

4.2 Discrete-Event State Machine

For each movement lane, the state stores an FCFS waiting queue, a busy/idle service indicator, arrival and departure counts, maximum waiting population, and the integral of waiting population over time. Events are arrival, green start/end,

and service completion. Service starts only if the lane is permitted and its two-second completion fits inside green. A completion at green end is valid. Clearance admits no new service. Equal-time events are ordered completion, green end, green start, arrival, then insertion order.

At each event, the engine first integrates the unchanged queue lengths since the prior event, then applies the state transition. The conservation invariant for each lane m is

$$A_m(t) = D_m(t) + Q_m(t) + S_m(t), \qquad m = 1, \ldots, 12, \tag{4.4}$$

where S_m is 0 or 1 for in-service population. A scheduled completion beyond the horizon is not counted as a departure by the horizon.

4.3 Objective and Complete Formulation

Let $g = (g_N, g_E, g_S, g_W)$ be integer-second green durations, C the cycle, L total clearance, T observation horizon, and ω a generated traffic scenario. Waiting population excludes vehicles in service. The single-run objective is

$$D_T(g, \omega) = \sum_{m=1}^{12} \int_0^T Q_m(t; g, \omega)\, dt \quad \text{vehicle-seconds.} \tag{4.5}$$

The finite-horizon OR formulation is

$$\min_{g \in \Omega} \quad \mathbb{E}_\omega[D_T(g, \omega)] \tag{4.6}$$

$$\Omega = \{g \in \mathbb{Z}^4 : \quad \sum_i g_i = C - L, \quad g_{\min} \le g_i \le g_{\max}, \tag{4.7}$$

$$g_i = g_{\min} + k_i \Delta, \quad k_i \in \mathbb{Z}_{\ge 0}\}. \tag{4.8}$$

The entered plan need not lie on the search lattice, although it must satisfy base timing constraints. In the reference case $C - L = 80$ seconds, $g_{\min} = 8$ seconds, and $g_{\max} = 44$ seconds. The four-second request has 415 feasible timings. A one-second request has 19,369.

4.4 Performance Measures

Between events, queue area increases by $Q_m(t_{old})(t_{new} - t_{old})$. Mean queue is area/$T$; throughput counts completed departures, and backlog counts vehicles still waiting at T. Mean waiting per arrival averages D_T/A across runs, including partial waits at T rather than completed-trip travel time.

Chapter 5

Implementation

5.1 Rust/WASM Engine and Experiment Pipeline

The Rust core separates deterministic queue transitions, scenario generation, and experiment orchestration. It generates seeded Poisson scenarios, enumerates feasible timings against common training arrivals, then evaluates the selected and reference plans on matched fresh arrivals.

The Rust engine has four crates: replaceable random generation, queue simulation, experiment orchestration, and a WebAssembly adapter. Each event applies $S_{n+1} = F(S_n, e_n, c)$ to explicit state and configuration, returning the next state without external I/O. The experiment session is a state machine that advances through training, fresh evaluation, optional sensitivity analysis, and completion.

The request fixes the 64-bit linear congruential generator seed. The same request and engine build reproduce the scenarios, and candidate plans replay the same arrivals for fair comparison. Batch size does not change the result. This supports audit and replay, but does not establish statistical independence or validate assumed traffic rates.

The Rust model compiles to the browser's `wasm32-unknown-unknown` target through `wasm-bindgen` [5, 6]. A worker advances bounded batches and reports progress to SolidJS. Large searches (at least 20,000 candidate–scenario evaluations) can split candidate ranges across up to 16 dedicated Web Workers, each running its own WASM instance; the coordinator merges statistics and one worker performs fresh evaluation. This is worker-level parallelism, not shared-memory WASM threading [7]. Graphs and animation display engine outputs and do not define the objective.

Batching exposes live progress. Queue profiles, moments, and density grids

are accumulated without retaining every vehicle trace; selected traces can be regenerated for playback.

5.2 Optimization and Application

After validating the request, the engine enumerates feasible g, scores each plan on common training scenarios, selects the lowest sample mean D_T, then compares the selected and reference plans on matched fresh scenarios. It reports timing, total and per-arrival waiting, queue profiles, residual backlog, and uncertainty. SolidJS renders SVG graphs and lane-correct replay from WASM results; the animation does not alter the objective. The public application is `https://traffic-signal-analysis.pages.dev/` [8].

For paired comparison, a replication's saving is

$$d_r = D_T(g_{\text{reference}}, \omega_r) - D_T(g_{\text{selected}}, \omega_r). \tag{5.1}$$

Positive d_r favors the selected plan. Welford's updates accumulate means and sample variances. A pointwise approximate 95% interval is

$$\bar{d} \pm t_{0.975,R-1} \frac{s_d}{\sqrt{R}}. \tag{5.2}$$

The paired-mean interval follows the usual Student-t form [1]. These intervals describe sampling variation conditional on the model. They do not measure uncertainty about field conditions or give a simultaneous guarantee over all searched timings.

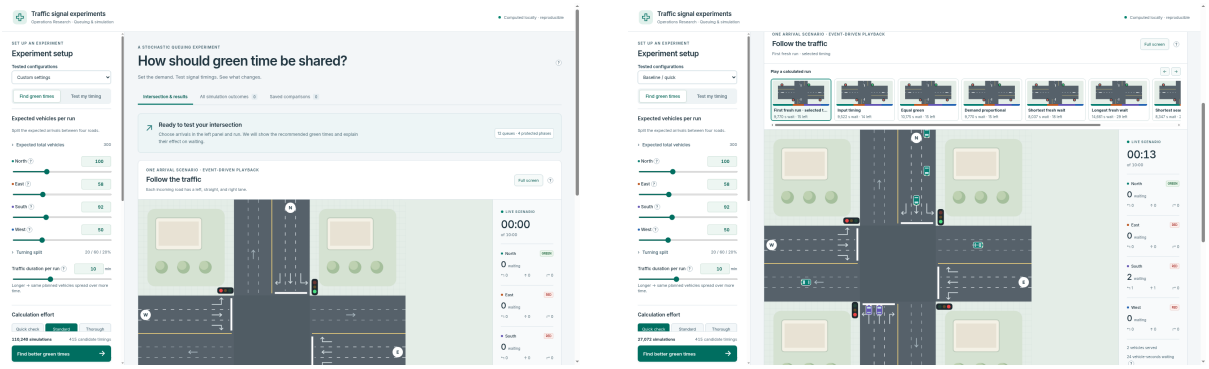


Figure 5.1: Browser application: scenario controls (left) and signal/vehicle replay (right)

Chapter 6

Testing

6.1 Verification and Monte Carlo Convergence

Tests cover inverse sampling, hand-checked event traces, phase boundaries, lane permissions, vehicle conservation, queue-area integration, grid enumeration, paired statistics, batch invariance, progress, and bounded summaries. An audit checked 23 saved outputs for request/replication consistency and agreement between queue profiles and cumulative waiting.

These checks establish consistency with the model, not field validity. No observed traffic was used to calibrate the assumed Poisson arrivals, turn probabilities, two-second headway, or protected phases. With repeated scenarios, the sample mean approaches the model expectation; individual run delays need not be Gaussian. More runs reduce sampling error but cannot fix incorrect assumptions. Figure 6.1 shows the running mean for 1,000 fresh runs.

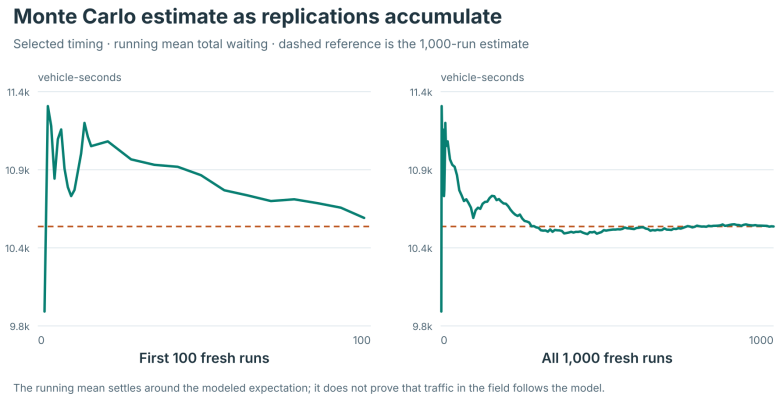


Figure 6.1: Running mean cumulative waiting over 1,000 fresh reference runs

Chapter 7

Results and Discussion

7.1 Reference Ten-Minute Result

At the baseline synthetic rates, the four-second grid contains 415 feasible timings. Each was scored on 256 shared training scenarios. The selected 28/16/24/12-second split was evaluated with 1,000 fresh scenarios, together with three reference plans. Table 7.1 gives the principal result.

Table 7.1: Principal fresh-evaluation result, ten-minute reference case

Measure	Value
Selected green N/E/S/W	28/16/24/12 s
Mean total waiting	10,507.8 vehicle-s/run
Pointwise 95% interval	10,430.7–10,584.9 vehicle-s/run
Mean observed waiting per arrival	35.08 s
Mean end waiting population	18.77 vehicles
Input minus selected paired saving	104.2 vehicle-s/run
Paired 95% interval	69.6–138.8 vehicle-s/run

The selected allocation equals the grid-projected demand-proportional baseline. Its gain over the entered timing is statistically resolved within this model but modest: about 0.35 seconds per expected arrival. The objective value is cumulative vehicle-seconds, not vehicles per second and not average seconds per completed trip.

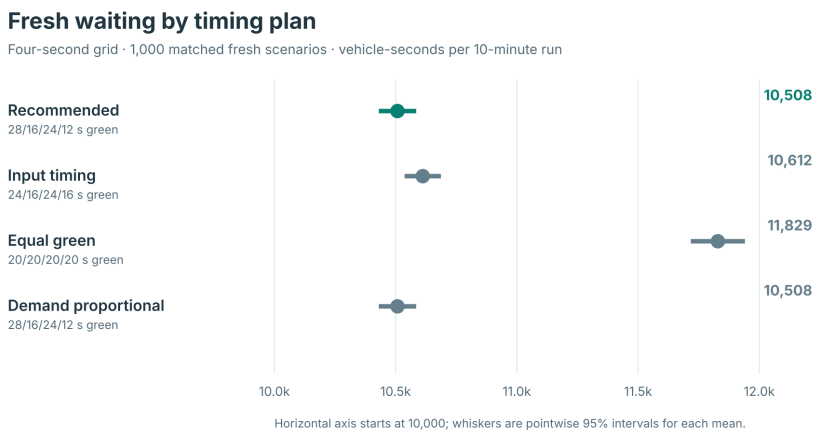


Figure 7.1: Fresh mean waiting for selected and reference timings; lower is better

7.2 Mean Queue Behavior

Figure 7.2 displays the mean waiting population by incoming approach over 1,000 fresh runs at the selected timing. Each point represents a short interval average, not one arbitrary example run. The combined area of the four curves equals the fresh mean total waiting within numerical precision. Vehicles still waiting at 600 seconds have contributed partial waiting.

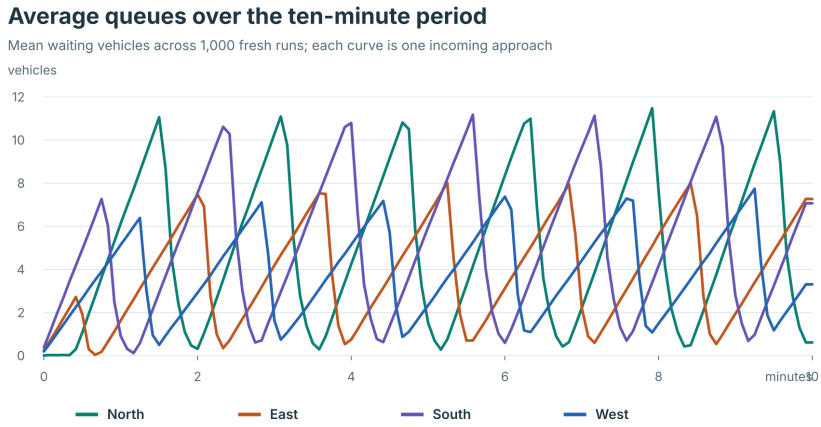


Figure 7.2: Average waiting queues by approach over the ten-minute period

7.3 Fine-Grid Search and Runtime

The one-second grid contains 19,369 feasible allocations. The original native study search used 128 training runs per candidate and finished in 209.02 seconds. The optimized Rust engine reran the same request in 6.86 seconds (30.5 times faster) and selected the same timing, 26/16/24/14 seconds. A separate

5,000-run evaluation gave 10,495.0 vehicle-seconds/run, with a pointwise 95% interval of 10,462.8–10,527.2. On the same 5,000 fresh scenarios, it saved 44.8 vehicle-seconds/run over the four-second-grid selection (paired 95% interval 34.8–54.8), or 0.42% of the coarse plan's mean. The speedup comes from reusing independent approach/phase simulations; it does not change the modeled objective.

Three additional matched seed checks each used 5,000 scenarios and favored the finer fixed plan, with savings of 46.4, 46.8, and 56.0 vehicle-seconds/run. The searches had different training counts; the result supports a small advantage of these two fixed plans for this ten-minute model, not proof of a global expected-value optimum.

Table 7.2 preserves the original-study wall-clock times for each complete main-study request when executed by the native release-mode Rust study program on the study machine. A request includes its timing search, fresh evaluation, and configured sensitivity work; the figures are not per-simulation costs. The optimized backend timings above are a separate post-study comparison. Neither set benchmarks the browser's WebAssembly workers or should be extrapolated to other hardware or to a million-replication run.

Table 7.2: Original-study native runtime for each principal study request

Study case	Grid	Search/fresh runs	Runtime (s)
Baseline quick	4 s	64/128	2.45
Baseline reference	4 s	256/1,000	10.27
Baseline precise	4 s	256/5,000	11.39
Quiet reference	4 s	256/1,000	4.92
Heavy reference	4 s	256/1,000	14.67
South-heavy reference	4 s	256/1,000	9.54
Fine-grid verification	1 s	128/5,000	209.02

The separate matched comparison of the coarse and fine selected plans took 1.25 seconds natively. The additional seed and horizon audit checks were not individually timed in the retained records, so no runtimes are claimed for them. These times are computational costs, not optimization objective values; waiting results remain the basis for the OR decision.

7.4 Demand, Capacity, and Horizon Sensitivity

Table 7.3 reports separately re-optimized synthetic scenarios. South-heavy demand keeps the total rate unchanged but shifts allocation toward South. Di-

rect cumulative-waiting comparisons across 150, 300, and 450 expected arrivals also change workload, so they are not pure timing benefits.

Table 7.3: Re-optimized four-second-grid demand scenarios

Scenario	Exp. arrivals	Green N/E/S/W	Waiting (veh-s)	End backlog
Quiet	150	36/12/20/12	4,580.6	8.27
Reference	300	28/16/24/12	10,507.8	18.77
Heavy	450	28/16/24/12	23,431.5	54.40
South-heavy	300	16/16/36/12	10,077.9	18.50

At the selected reference split, heavy straight demand is 540, 315, 495, and 270 vehicles/hour for North, East, South, and West. Ideal per-lane capacities are 525, 300, 450, and 225 vehicles/hour, respectively.

Even without integer rounding, those four straight streams require 86.4 green seconds per cycle, more than the available 80. No feasible reallocation can serve all heavy straight demand under this model. Mean end backlog grows from 54.4 vehicles at ten minutes to 110.1 at 32 minutes. This is a structural capacity limit, not a failure of timing search.

The principal 600-second horizon ends after 6.25 signal cycles. Its endpoint and empty initial queues are part of the objective. Re-optimization selected 28/16/24/12 seconds at 576, 600, 624, 672, and 960 seconds, but 24/16/24/16 seconds at 1,920 seconds. To isolate the timing effect, 5,000 fresh matched scenarios per horizon compared the two fixed plans (seed 2027).

Table 7.4: Paired saving of 28/16/24/12 s against 24/16/24/16 s

Horizon	Mean saving (vehicle-s/run)	Paired 95% interval
600 s	105.5	88.5–122.5
960 s	64.0	35.8–92.2
1,920 s	15.0	−40.3–70.2

The 32-minute interval includes both signs; there is no resolved advantage at that horizon. The recommendation must therefore remain specific to the ten-minute experiment. Total vehicle-seconds from unequal horizons cannot be compared directly as if the exposure were identical.

Chapter 8

Conclusion and Future Scope

8.1 Conclusion

In the ten-minute synthetic case, the four-second search selected 28/16/24/12 seconds; mean waiting was 10,507.8 vehicle-seconds per run. It tied the demand-based plan and modestly improved on the entered timing. The finer grid improved mean waiting by 0.42%. Under heavy demand, modeled capacity was exceeded; at 32 minutes, the timing advantage was unresolved.

8.2 Limitations

Results depend on synthetic Poisson rates, fixed turns, dedicated lanes, deterministic service, protected phases, empty initial queues, and the chosen horizon. No field calibration was performed; intervals quantify simulation variation conditional on these assumptions and do not prove real-world optimality.

8.3 Future Scope

1. Collect approach counts, turn proportions, headways, and observed queues, then calibrate against held-out periods.
2. Extend the model to platoons, pedestrians, shared lanes, downstream storage, and alternative phase structures.
3. Validate predicted queues and waiting against field observations before making operational recommendations.

References

[1] H. A. Taha, *Operations Research: An Introduction*, 10th ed. Pearson, 2017. Online.

[2] T. L. Saaty, *Elements of Queuing Theory, with Applications*. New York: McGraw–Hill, 1961. Online.

[3] J. D. C. Little, "A Proof for the Queuing Formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. Online.

[4] Federal Highway Administration, *Traffic Signal Timing Manual*, Chapter 3, FHWA-HOP-08-024. Online.

[5] Rust Project, *The rustc book: wasm32-unknown-unknown Target*. Online.

[6] Rust WebAssembly Working Group, *The wasm-bindgen Guide*. Online.

[7] WHATWG, *HTML Living Standard: Web Workers*. Online.

[8] Project team, *Traffic Signal Analysis: Interactive Simulation Application*, 2026. Online.